

Flappy Goose

Relazione del Progetto

Abstract— Questo documento descrive la progettazione e lo sviluppo di "Flappy Goose". La relazione include l'analisi dei requisiti funzionali e non funzionali, le scelte progettuali, e una panoramica delle tecnologie utilizzate per realizzare il progetto. Il gioco combina una logica di gioco fluida, una simulazione fisica e una grafica 2D accattivante. L'obiettivo è offrire un gameplay stimolante che sfida i riflessi del giocatore.

I. INTRODUZIONE

Flappy Goose è un clone spirituale del popolare gioco Flappy Bird, creato utilizzando Processing, un software flessibile per schizzi e linguaggio per le arti visive e il design. Questo progetto mira a ricreare il gameplay avvincente e la meccanica semplice dell'originale, aggiungendo un tocco personale attraverso grafica e gameplay personalizzati.

II. ANALISI DEI REQUISITI

A. Requisiti Funzionali

- **Controllo dell'Oca:** Il giocatore deve poter far saltare l'oca premendo spazio.
- **Generazione degli Ostacoli:** Gli ostacoli (meteoriti) devono essere generati in modo casuale con spaziatura variabile per garantire una sfida dinamica.
- **Sistema di Punteggio:** Il punteggio totale viene calcolato rispetto al tempo di sopravvivenza.
- **Gestione delle Collisioni:** Il gioco termina quando l'oca collide con un meteorite o tocca il terreno.
- **Schermata di Fine Gioco:** Dopo la collisione, deve essere mostrato un messaggio di "Game Over" con il punteggio totale.

B. Requisiti Non Funzionali

- **Prestazioni:** Il gioco deve mantenere un framerate stabile di almeno 60 FPS per garantire una giocabilità fluida.
- **Usabilità:** L'interfaccia deve essere semplice e intuitiva, adatta a giocatori di tutte le età.
- **Compatibilità:** Il sistema deve essere eseguibile su piattaforme diverse (Windows, macOS, Linux).
- **Manutenibilità:** Il codice deve essere modulare e ben documentato per facilitare futuri miglioramenti.

III. SCELTE PROGETTUALI

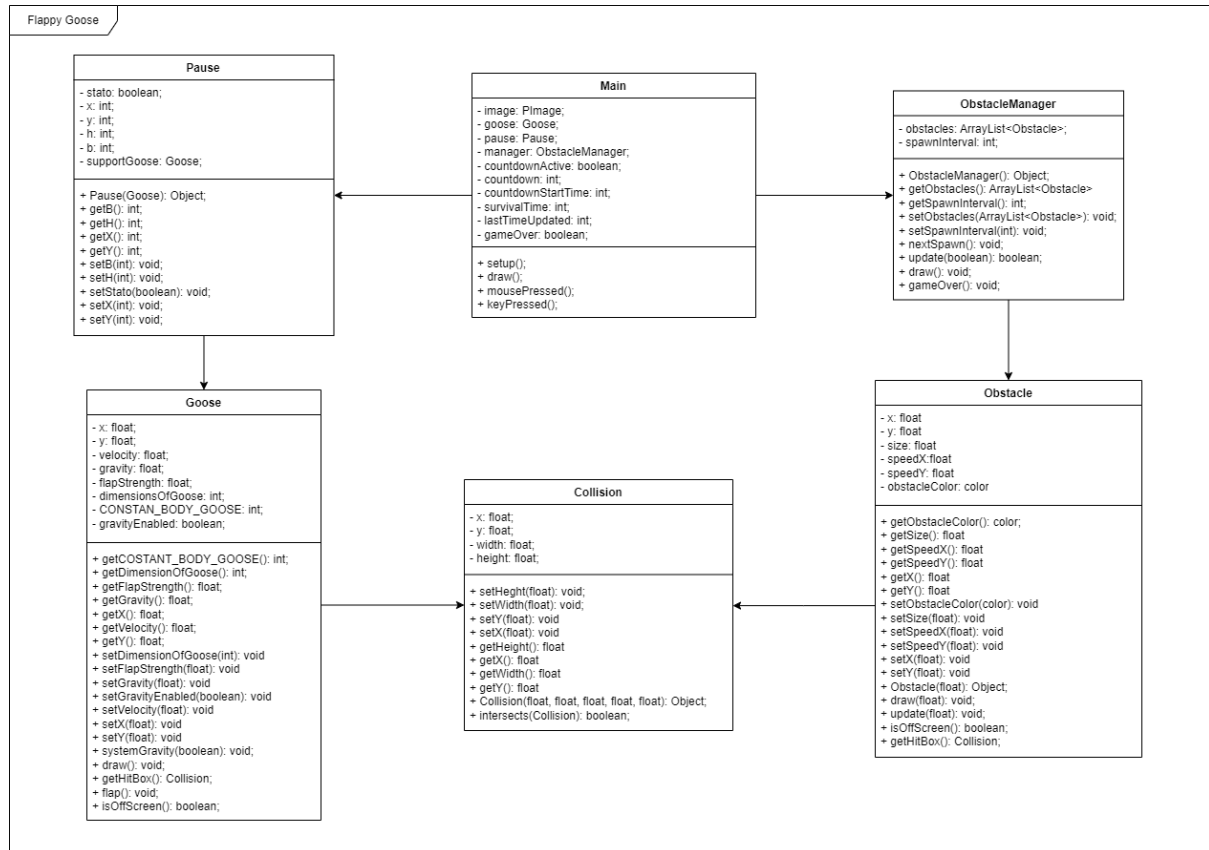
A. Componenti Principali e Responsabilità

- **Motore di Gioco:** Aggiorna le posizioni dell'oca e degli ostacoli a ogni ciclo del gioco; Verifica le collisioni e determina il termine della partita.
- **Generatore di Ostacoli:** Crea nuovi ostacoli con spazi casuali e li posiziona in sequenza.
- **Gestore del Punteggio:** Tiene traccia del punteggio e lo aggiorna quando l'oca supera un ostacolo.
- **Rilevatore di Collisioni:** Identifica il contatto tra l'oca e gli ostacoli o il terreno.
- **Gestione dell'Input:** Intercetta gli input del giocatore e li utilizza per far saltare l'oca.
- **Renderizzatore Grafico:** Disegna l'oca, gli ostacoli, il punteggio e i messaggi finali sullo schermo.

B. Tecnologie e Strumenti Utilizzati

- *Linguaggio di Programmazione:* Java.
- *Strumenti:* Processing.

IV. DIAGRAMMA UML



[UML diagram link](#)